

Clase 4 - MVC conectado a SQL Server

Objetivo de la clase

Al finalizar, el estudiante deberá poder conectar una aplicación ASP.NET MVC 5 a SQL Server y ejecutar un CRUD real sobre la tabla `Estudiantes`.

Resultado esperado al finalizar la clase:

`ASP.NET` → `EstudiantesController` → `Entity Framework` → `SQL Server` → `Tabla Estudiantes`

Dejaremos de Utilizar:

```
public static List<Estudiante> estudiantes
```

¿Qué problema tiene nuestra lista?

```
public static List<Estudiante> estudiantes = new List<Estudia
nte>()
{
    new Estudiante
    {
        Id = 1,
        Nombre = "Juan Pérez",
        Correo = "jp@mail.com"
    },
    new Estudiante
    {
        Id = 2,
        Nombre = "María López",
        Correo = "ml@mail.com"
```

```
    }
};
```

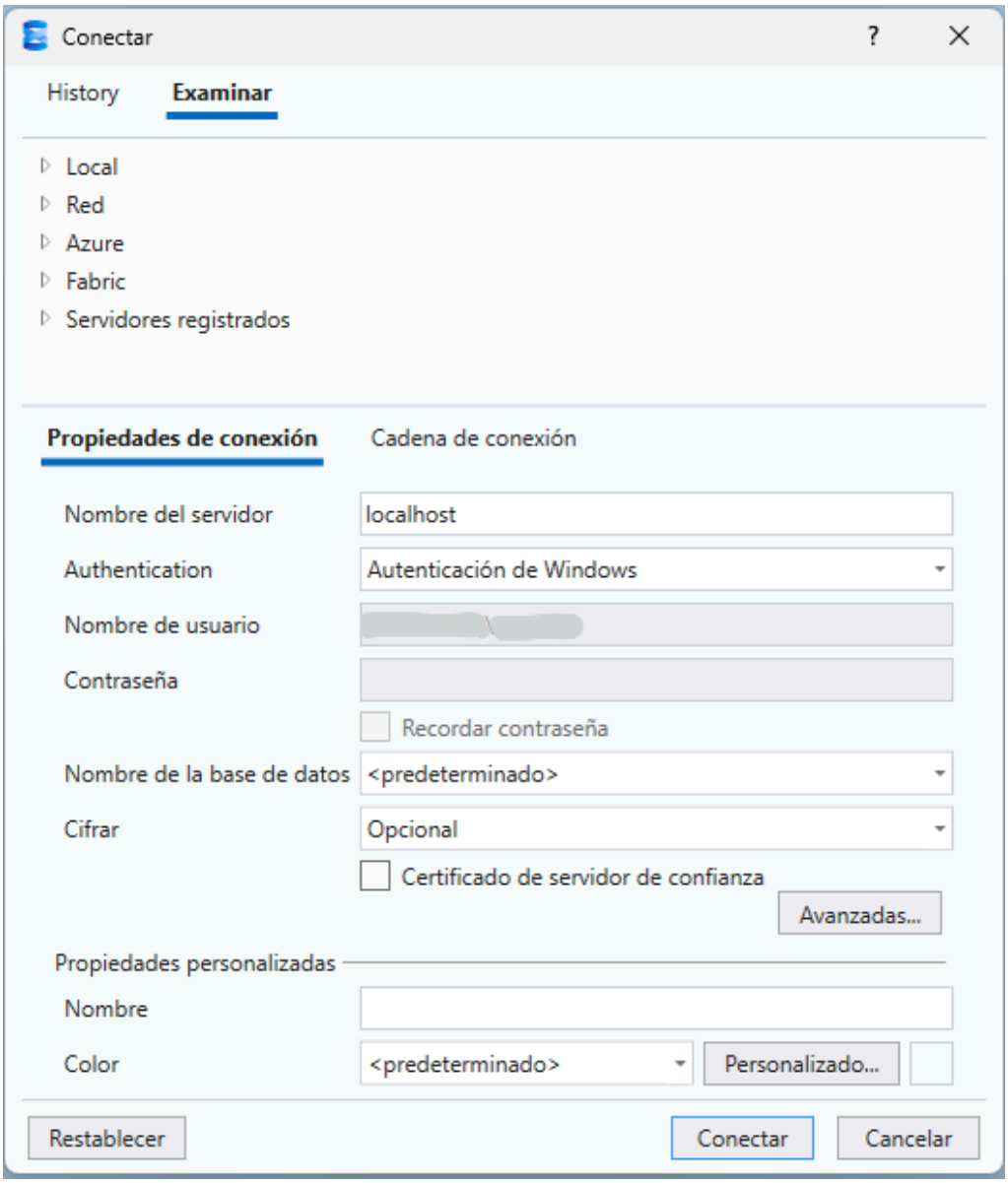
Esto ha sido muy útil para aprender MVC pero, **¿Qué ocurre si cerramos la aplicación?**

- Los datos nuevos desaparecen.

Practica de la clase:

Crear la base de datos

Abriremos **SQL Server Management Studio** y establecemos la conexión con nuestra instancia local de SQL Server.


Conectar
History
Examinar
Local
Red
Azure
Fabric
Servidores registrados
Propiedades de conexión
Cadena de conexión
Nombre del servidor
localhost
Authentication
Autenticación de Windows
Nombre de usuario
Contraseña
Recordar contraseña
Nombre de la base de datos
<predeterminado>
Cifrar
Opcional
Certificado de servidor de confianza
Avanzadas...
Propiedades personalizadas
Nombre
Color
<predeterminado>
Personalizado...
Restablecer
Conectar
Cancelar



En la pestaña de consultas, vamos a ejecutar el siguiente código:

```
CREATE DATABASE SistemaAcademicoDB;
GO

/*
    Es importante para que lo que sigue en nuestro siguiente codigo T-SQL
    se ejecute en la base SistemaAcademicoDB y no en master u otra base de datos
    que esté en uso
*/
USE SistemaAcademicoDB;
GO

CREATE TABLE Estudiantes
(
    Id INT IDENTITY(1,1) PRIMARY KEY, // SQL Server generará automáticamente el Id en incrementos de 1 en 1
```

```
    Nombre NVARCHAR(100) NOT NULL,
    Correo NVARCHAR(150) NOT NULL
);

// Insertar datos iniciales
INSERT INTO Estudiantes (Nombre, Correo)
VALUES ('Juan Pérez', 'jp@mail.com');

INSERT INTO Estudiantes (Nombre, Correo)
VALUES ('María López', 'ml@mail.com');

INSERT INTO Estudiantes (Nombre, Correo)
VALUES ('Carlos García', 'cg@mail.com');

// Comprobar datos insertados
SELECT * FROM Estudiantes;
```

Resultado:

	Id	Nombre	Correo
1	1	Juan Pérez	jp@mail.com
2	2	María López	ml@mail.com
3	3	Carlos García	cg@mail.com

¿Qué vamos a utilizar para conectarnos?

Entity Framework nos permite trabajar con datos de una base de datos utilizando objetos y clases de C#.

Sin **Entity Framework** pensaríamos principalmente en:

`SELECT`
`INSERT`
`UPDATE`
`DELETE`

Con **Entity Framework** trabajaremos con:

```
db.Estudiantes.ToList();

db.Estudiantes.Add(estudiante);

db.Estudiantes.Remove(estudiante);

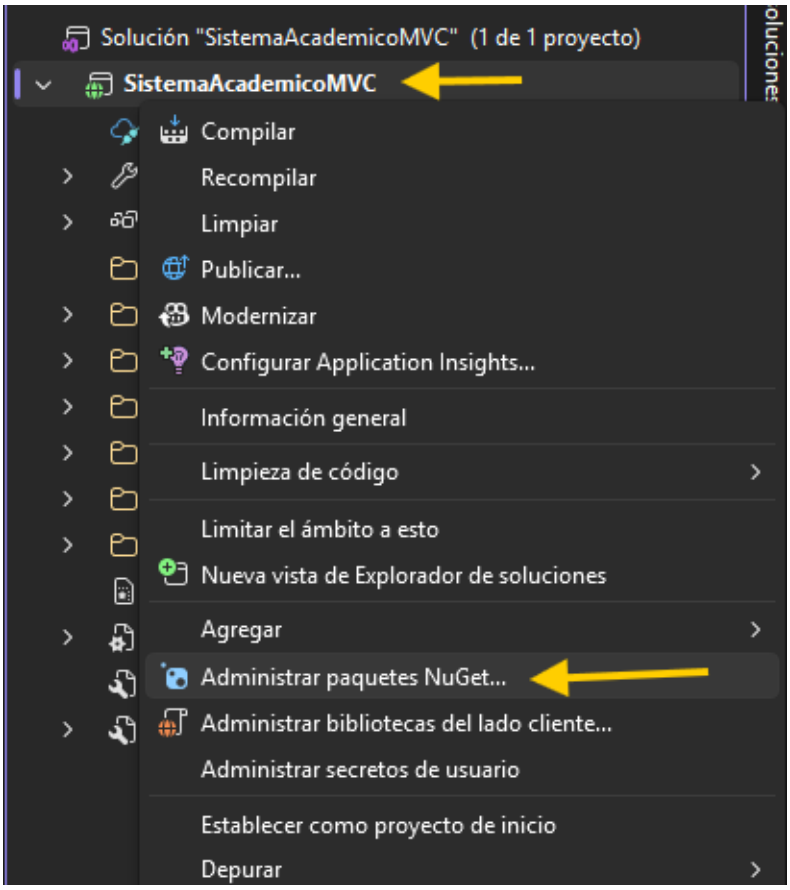
db.SaveChanges();
```

Entity Framework actúa como intermediario entre nuestros objetos C# y la base de datos.

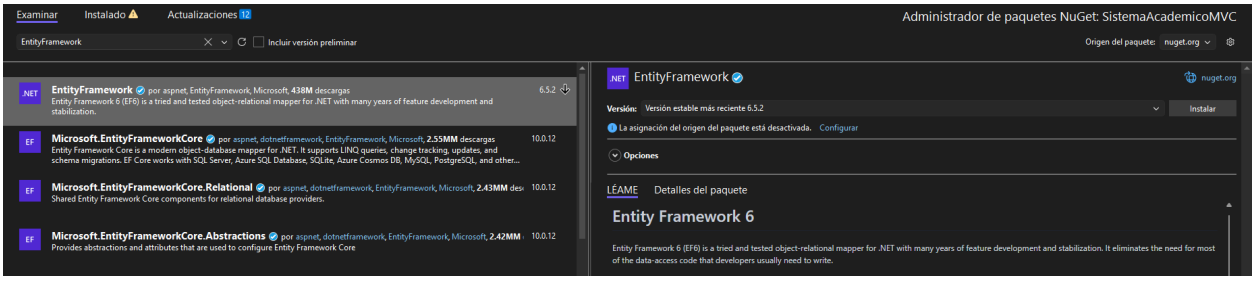
ASP.NET MVC 5 suele trabajar perfectamente con **Entity Framework.**

Abrirmos el proyecto `SistemaAcademicoMVC` que se encuentra en el aula de virtual

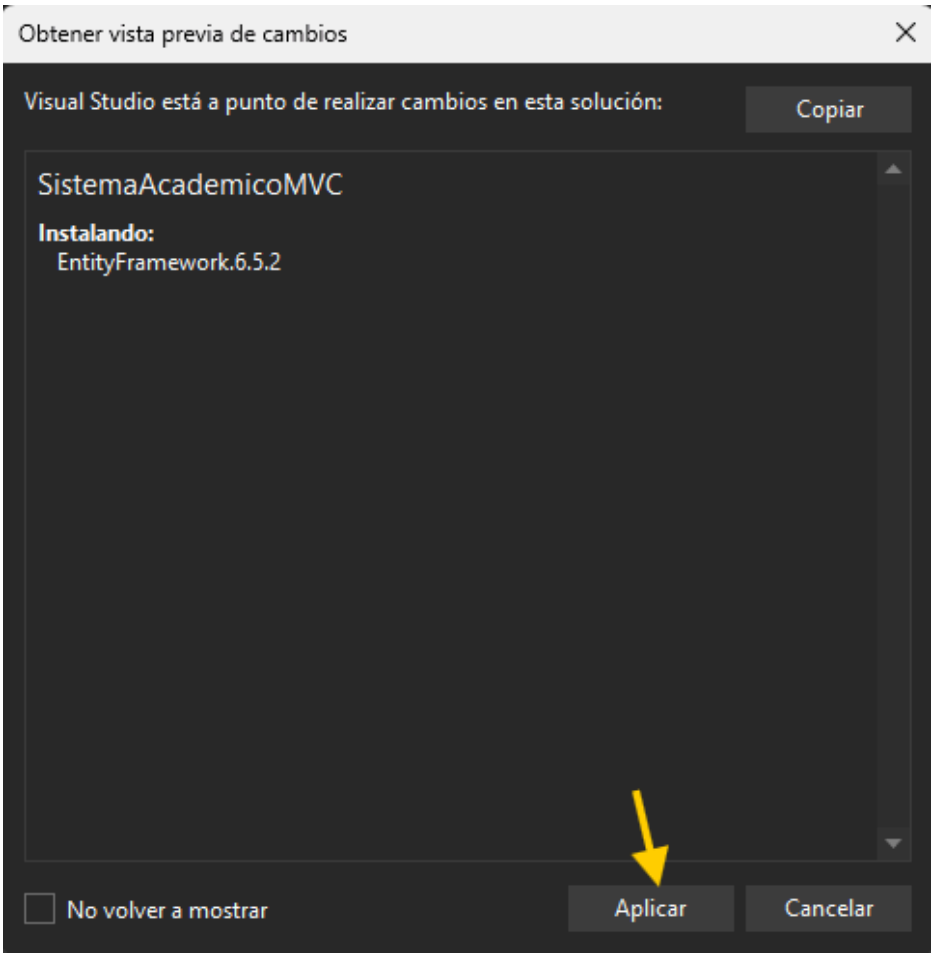
Procedemos a instalar el paquete de **Entity Framework.**

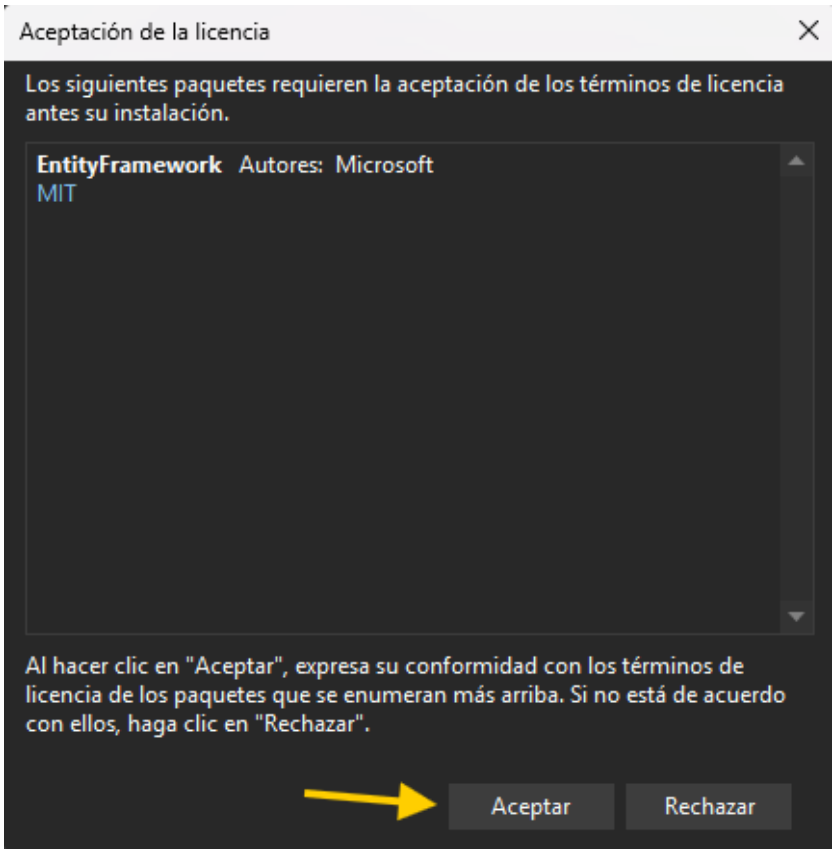


Nos aparecerá una pestaña como esta:



Buscaremos la palabra `EntityFramework` En el apartado de **Examinar**, luego seleccionamos la primera opción y presionamos el botón de **instalar**.


Obtener vista previa de cambios
Visual Studio está a punto de realizar cambios en esta solución:
Copiar
SistemaAcademicoMVC
Instalando:
EntityFramework.6.5.2
No volver a mostrar
Aplicar
Cancelar

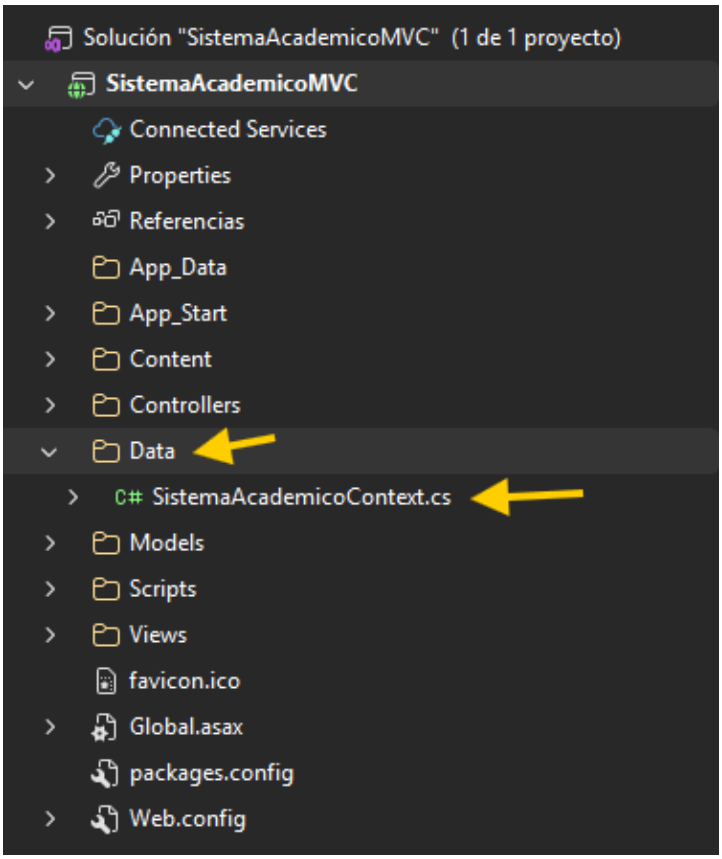


Nuestro Model prácticamente no cambia

```
C#                      SQL Server

Estudiante              Estudiantes
-----------             -----------
Id          ←→          Id
Nombre      ←→          Nombre
Correo      ←→          Correo
```

Ahora en el Explorador de Soluciones agregaremos una carpeta llamada **Data** y dentro de ella una clase llamada **SistemaAcademicoContext.cs**



La clase **SistemaAcademicoContext.cs** que acabamos de crear, debe quedar con el siguiente código:

```
using SistemaAcademicoMVC.Models;
using System.Data.Entity;

namespace SistemaAcademicoMVC.Data
{
    public class SistemaAcademicoContext : DbContext
    {
        static SistemaAcademicoContext()
        {
            Database.SetInitializer<SistemaAcademicoContext>
(null);
        }

        public SistemaAcademicoContext()
```

```
            : base("SistemaAcademicoDB")
        {
        }

        public DbSet<Estudiante> Estudiantes { get; set; }
    }
}
```

- **DbContext** representa nuestra conexión/contexto de trabajo con la base.
- **DbSet** Tabla Estudiantes.

```
SistemaAcademicoContext

     │
     └── DbSet<Estudiante>
            │
            ▼
       Tabla Estudiantes
```

Nuestra base ya existe. No queremos que Entity Framework intente crearla o modificar su estructura automáticamente por eso se coloca.

```
static SistemaAcademicoContext()
{
    Database.SetInitializer<SistemaAcademicoContext>(null);
}
```

Ahora abrimos el web.config de nuestro proyecto y dentro de el tag `<configuration>` después de
`<configSections>` agregaremos nuestra cadena de conexión.

```
<connectionStrings>
    <addname="SistemaAcademicoDB"
connectionString="Server=localhost;Database=master;Trusted_Connection=True;"
providerName="System.Data.SqlClient" />
</connectionStrings>
```

Abrimos **EstudiantesController** y lo dejamos de la siguiente manera:

```
using SistemaAcademicoMVC.Models;
using SistemaAcademicoMVC.Data;
using System.Linq;
using System.Web.Mvc;

namespace SistemaAcademicoMVC.Controllers
{
    public class EstudiantesController : Controller
    {
        private readonly SistemaAcademicoContext db =
            new SistemaAcademicoContext();

        // GET: Estudiantes
        [HttpGet]
        public ActionResult Index()
        {
            var estudiantes = db.Estudiantes.ToList();
            return View(estudiantes);
        }

        //agregar un nuevo estudiante
        [HttpPost]
        public ActionResult Create(Estudiante estudiante)
        {
            db.Estudiantes.Add(estudiante);

            db.SaveChanges();

            return RedirectToAction("Index");
        }

        //Details de un estudiante
        [HttpGet]
        public ActionResult Details(int id)
```

```
        {
            var estudiante = db.Estudiantes.Find(id);
            if (estudiante == null)
            {
                return HttpNotFound();
            }
            return View(estudiante);
        }

        //Editar un estudiante
        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult Edit(int id, Estudiante estudiant
e)
        {
            var estudianteExistente = db.Estudiantes.Find(i
d);
            if (estudianteExistente == null)
            {
                return HttpNotFound();
            }
            estudianteExistente.Nombre = estudiante.Nombre;
            estudianteExistente.Correo = estudiante.Correo;

            db.SaveChanges();

            return RedirectToAction("Index");
        }

        //Eliminar un estudiante
        public ActionResult Delete(int id)
        {
            var estudiante = db.Estudiantes.Find(id);
            if (estudiante == null)
            {
                return HttpNotFound();
```

```
            }

            db.Estudiantes.Remove(estudiante);

            db.SaveChanges();

            return RedirectToAction("Index");
        }
    }
}
```

Correspondencia CRUD

CRUD	Operación	EF
Create	Crear	`Add()`
Read	Consultar	`ToList()` / `Find()`
Update	Actualizar	Modificar + `SaveChanges()`
Delete	Eliminar	`Remove()`

Todos terminan, cuando modifican información, con: `db.SaveChanges();`

Agregar un Login a nuestro Sistema

Agregaremos una nueva tabla a nuestra Base de Datos `SistemaAcademicoDB` desde **SQL Server Management Studio**

```
CREATE TABLE Usuarios
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Nombre NVARCHAR(100) NOT NULL,
    Correo NVARCHAR(150) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(500) NOT NULL,
```

```
    Activo BIT NOT NULL DEFAULT 1
);
```

Crear el Model Usuario

Agregamos una nueva clase a nuestro proyecto `SistemaAcademicoMVC`:

```
Models
└── Usuario.cs
```

La clase tendrá las siguientes propiedades:

```
using System.ComponentModel.DataAnnotations;

namespace SistemaAcademicoMVC.Models
{
    public class Usuario
    {
        public int Id { get; set; }
        [Required]
        public string Nombre { get; set; }
        [Required]
        [EmailAddress]
        public string Correo { get; set; }
        [Required]
        public string PasswordHash { get; set; }
        public bool Activo { get; set; }
    }
}
```

Agregar Usuarios al DbContext

Ingresar a la clase `SistemaAcademicoContext` y colocar el nuevo DbSet de Usuarios

`public DbSet<Usuario> Usuarios { get; set; }`

Agregar ViewModels

Para este ejemplo, agregaremos una carpeta al proyecto llamada ViewModels y dentro de ella agregaremos dos clases llamadas `LoginViewModel` y `CreateUserViewModel`

```
Models
└── LoginViewModel.cs
└── CreateUserViewModel.cs
```

```
using System.ComponentModel.DataAnnotations;

namespace SistemaAcademicoMVC.ViewModels
{
    public class LoginViewModel
    {
        [Required(ErrorMessage = "El correo es obligatorio")]
        [EmailAddress(ErrorMessage = "Ingrese un correo válid
o")]
        public string Correo { get; set; }

        [Required(ErrorMessage = "La contraseña es obligatori
a")]
        [DataType(DataType.Password)]
        public string Password { get; set; }
    }
}
```

```
using System.ComponentModel.DataAnnotations;

namespace SistemaAcademicoMVC.ViewModels
{
    public class CreateUserViewModel
    {
        [Required(ErrorMessage = "El nombre es obligatorio")]
```

```
        public string Nombre { get; set; }

        [Required(ErrorMessage = "El correo es obligatorio")]
        [EmailAddress(ErrorMessage = "Ingrese un correo válid
o")]
        public string Correo { get; set; }

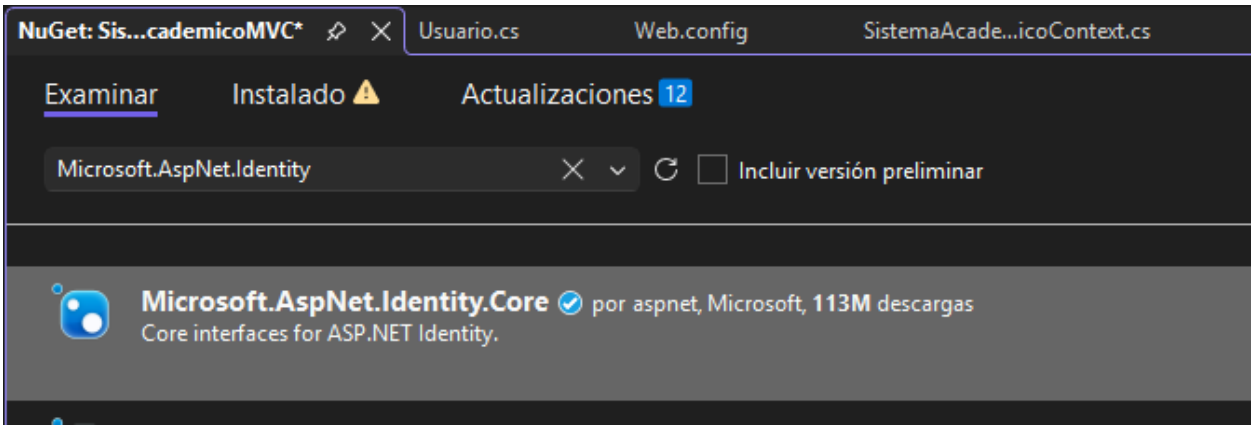
        [Required(ErrorMessage = "La contraseña es obligatori
a")]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [Required(ErrorMessage = "Debe confirmar la contraseñ
a")]
        [DataType(DataType.Password)]
        [Compare("Password", ErrorMessage = "Las contraseñas
no coinciden")]
        public string ConfirmarPassword { get; set; }
    }
}
```

Un **ViewModel** representa los datos que necesita específicamente una View.

Agregar Nuget.

Ahora en el Administrador de paquetes Nuget vamos a instalar `Microsoft.AspNet.Identity.Core`



Agregar Controller

Para lograr una opción de Login, tenemos que crear un nuevo controller llamado `AccountController`

```
using Microsoft.AspNet.Identity;
using SistemaAcademicoMVC.Data;
using SistemaAcademicoMVC.Models;
using SistemaAcademicoMVC.ViewModels;
using System.Linq;
using System.Web.Mvc;
using System.Web.Security;

namespace SistemaAcademicoMVC.Controllers
{
    public class AccountController : Controller
    {
        private readonly SistemaAcademicoContext db =
            new SistemaAcademicoContext();

        [HttpGet]
        public ActionResult Login()
        {
            return View();
```

```
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult Login(LoginViewModel model)
        {
            if (!ModelState.IsValid)
            {
                return View(model);
            }

            var usuario = db.Usuarios
                .FirstOrDefault(u =>
                    u.Correo == model.Correo &&
                    u.Activo);

            if (usuario == null)
            {
                ModelState.AddModelError("", "Correo o contra
seña incorrectos.");

                return View(model);
            }

            var hasher = new PasswordHasher();

            var resultado = hasher.VerifyHashedPassword(
                usuario.PasswordHash,
                model.Password
            );

            if (resultado == PasswordVerificationResult.Faile
d)
            {
                ModelState.AddModelError("", "Correo o contra
seña incorrectos.");
```

```
                return View(model);
            }

            FormsAuthentication.SetAuthCookie(usuario.Correo,
false);

            return RedirectToAction("Index", "Estudiantes");
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult Logout()
        {
            FormsAuthentication.SignOut();

            return RedirectToAction("Login");
        }

        [HttpGet]
        public ActionResult CreateUser()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public ActionResult CreateUser(CreateUserViewModel us
uario_)
        {
            if (!ModelState.IsValid)
            {
                return View(usuario_);
            }

            var existeCorreo = db.Usuarios
```

```
                .Any(u => u.Correo == usuario_.Correo);

            if (existeCorreo)
            {
                ModelState.AddModelError(
                    "Correo",
                    "Ya existe un usuario con este correo."
                );

                return View(usuario_);
            }

            var hasher = new PasswordHasher();

            var usuario = new Usuario
            {
                Nombre = usuario_.Nombre,
                Correo = usuario_.Correo,
                PasswordHash = hasher.HashPassword(usuario_.P
assword),
                Activo = true
            };

            db.Usuarios.Add(usuario);
            db.SaveChanges();

            return RedirectToAction("Login");
        }

        protected override void Dispose(bool disposing)
        {
            if (disposing)
            {
                db.Dispose();
            }
```

```
            base.Dispose(disposing);
        }
    }
}
```

¿Para qué sirve FormsAuthentication?

`FormsAuthentication.SetAuthCookie(usuario.Correo, false);`

Le indica a ASP.NET que este usuario ya inició sesión correctamente, ASP.NET genera una cookie de autenticación.

Agregar Vista.

Luego debemos agregar las vistas correspondientes para que el Login funcione correctamente, dentro de la carpeta Views vamos a agregar una subcarpeta llamada Account y dentro de ella agregaremos la vista `Login.cshtml`.

```
Views
└── Account
    └── Login.cshtml
```

```
@model SistemaAcademicoMVC.ViewModels.LoginViewModel

@{
    ViewBag.Title = "Iniciar sesión";
}

<h2>Iniciar sesión</h2>

@using (Html.BeginForm("Login", "Account", FormMethod.Post))
{
    @Html.AntiForgeryToken()
    @Html.ValidationSummary(true, "", new { @class = "text-da
```

```
nger" })

    <div class="form-group">
        @Html.LabelFor(model => model.Correo)
        @Html.TextBoxFor(
            model => model.Correo,
            new
            {
                @class = "form-control",
                type = "email"
            }
        )

        @Html.ValidationMessageFor(model => model.Correo, "",
new { @class = "text-danger" })
    </div>

    <br />

    <div class="form-group">
        @Html.LabelFor(model => model.Password)

        @Html.PasswordFor(
            model => model.Password,
            new
            {
                @class = "form-control"
            }
        )

        @Html.ValidationMessageFor(
            model => model.Password,
            "",
            new { @class = "text-danger" }
        )
    </div>
```

```
    <br />

    <button type="submit" class="btn btn-primary">Iniciar ses
ión</button>

    <br />

        <p>
            Si aún no estás registrado,
            @Html.ActionLink(
                "Ingresa aquí",
                "CreateUser",
                "Account"
            )
        </p>
}
```

Modificar archivo `web.config`.

Dentro del tag `<system.web>` vamos a agregar la siguiente configuración:

```
<authentication mode="Forms">
 <forms loginUrl="~/Account/Login" timeout="30" />
</authentication>
```

Ahora ASP.NET sabe que la página de login es: `/Account/Login`

Proteger EstudiantesController

Hasta ahora cualquier persona puede entrar a Estudiantes, ver y modificar los datos, pero con el atributo [Authorize] lograremos restringir el acceso a los usuarios no Autenticados.

Por lo tanto tenemos que ingresar a `EstudiantesController` y hace una pequeña pero importante modificación.

```
[Authorize]
public class EstudiantesController : Controller
```

Modificar `_Layout.cshtml` para agregar enlace de Cerrar sesión.

Debemos agregar el siguiente codigo como una opción de la barra de navegación de nuestro Layout.

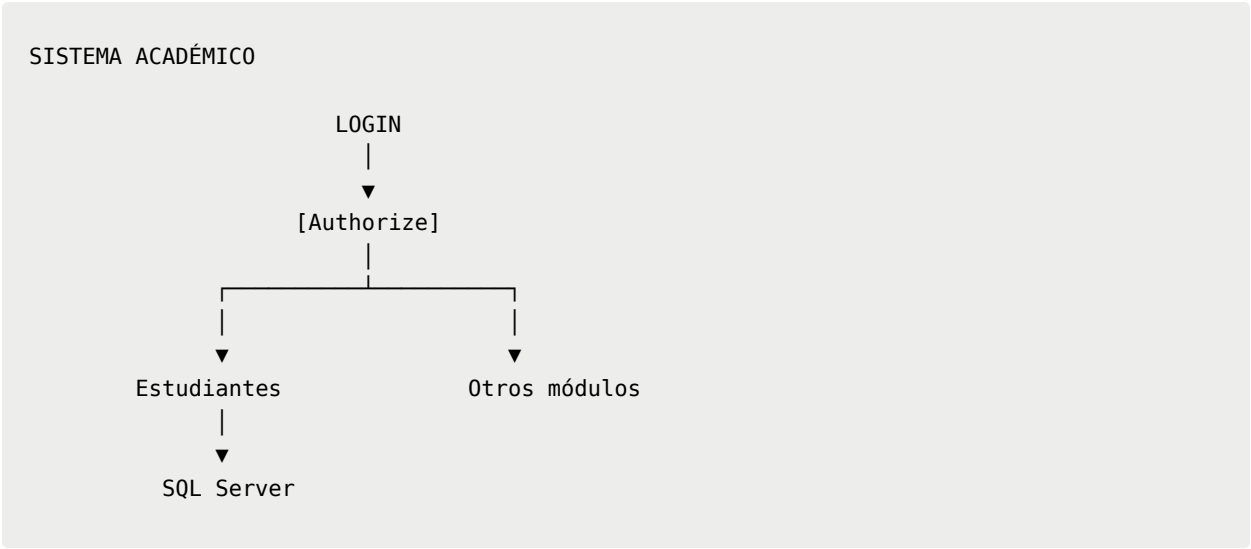
```
<li>
    @if (User.Identity.IsAuthenticated)
    {
        using (Html.BeginForm(
            "Logout",
            "Account",
            FormMethod.Post,
            new { id = "logoutForm" }))
        {
            @Html.AntiForgeryToken()

            <button type="submit"
                    class="btn btn-link">

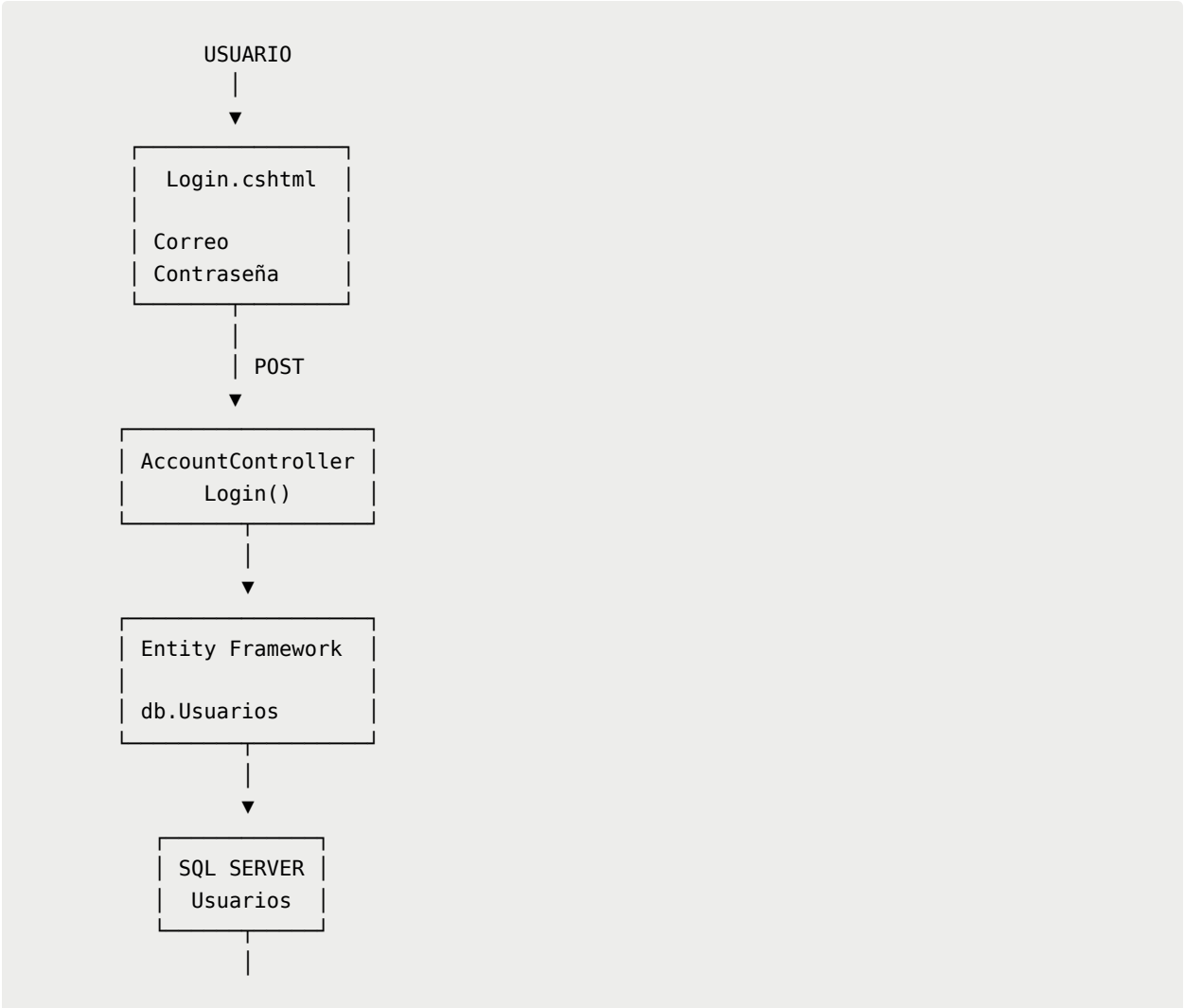
                Cerrar sesión

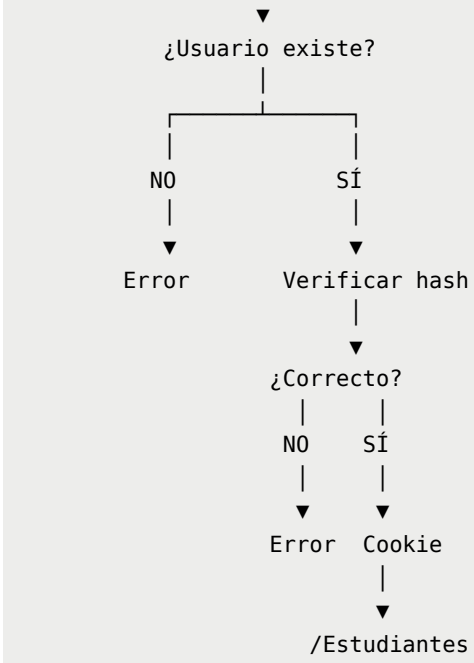
            </button>
        }
    }
</li>
```

Como está funcionando nuestro sistema:



Como está funcionando nuestro Login:




¿Usuario existe?
NO
SÍ
Error
Verificar hash
¿Correcto?
NO
SÍ
Error
Cookie
/Estudiantes